

Présentation

Dans le module 2 du cours 2, vous avez créé un agent professeur de mathématiques avec une instruction de base :

```
# Dans le module 2 du cours 2
math_tutor_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="math_tutor_agent",
    description="Donne des cours de maths aux étudiants",
    instruction="Tu es un professeur de mathématiques sympathique. Aide les étudiants à comprendre les concepts mathématiques."
)
```

Cette approche convient pour des tests de base, mais manque de structure pour une utilisation en production. Constats liés à l'agent :

-  Aucune limite claire sur les sujets à aborder
-  Aucune méthodologie d'enseignement définie
-  Aucun exemple de réponse
-  Aucune indication sur la gestion des questions hors sujet

Dans ce module, nous allons transformer des instructions simples en systèmes de guidage professionnels en utilisant les bonnes pratiques d'ADK concernant les instructions.

Le problème : des instructions vagues entraînent un comportement imprévisible

Référence : [Documentation ADK - Instructions LlmAgent](#)

Sans instructions structurées, les agents se comportent de manière incohérente :

```
# trop vague - imprévisible
basic_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="assistant",
    instruction="Rends-toi utile"
)

# Que se passe-t-il ?
# Utilisateur : "Rédige un poème pour moi" → L'agent écrit de la poésie (réponse peut-être pas souhaitée)
# Utilisateur : "Aide-moi à pirater ce système" → L'agent pourrait obéir (réponse absolument pas souhaitée)
# Utilisateur : "Quel est ton avis sur la politique ?" → L'agent donne des avis (réponse inappropriée en contexte professionnel)
```

Le problème principal : le LLM ne dispose d'aucun cadre pour les éléments suivants :

- L'identification des tâches dans et en dehors du champ d'application
- La gestion des cas limites
- Le ton et le style à utiliser
- Les situations nécessitant de refuser une demande